

Problem I

Best Position to Deploy Skadi

Time limit: 3 seconds

Memory limit: 1024 megabytes

Problem Description

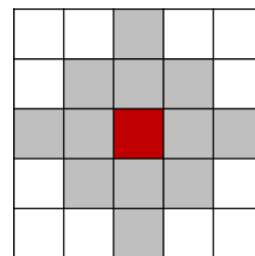
Molis is playing a tower defense game. Most of the defending operators have already been deployed. Now, only one support operator remains – Skadi.

Skadi can greatly enhance the attack and defense of nearby operators. Therefore, Molis wants to place her in a position where she can support as many operators as possible.

However, before he could finish this crucial step, Molis was called away by his mom to take out the trash. With the enemy fast approaching, he turns to you for help.

Please write a program to determine one position where Skadi can be placed to support the maximum number of already-deployed operators. Skadi's support range covers:

- The 8 adjacent cells around her (in all directions), and
- The 4 cells two steps away in the cardinal directions (up, down, left, right).



In total, Skadi supports operators located in the 12 surrounding cells. See the diagram for a visual representation of her range (Skadi is in red, support range in gray).

Input Format

The first line of the input are two integers m and n , representing the number of rows and columns of the map ($1 \leq m, n \leq 1,000$). Followed by m lines, each containing a string of length n , representing the map: '*' indicates a cell with a deployed operator, where Skadi cannot be placed on these cells. And '.' indicates an empty cell. It is guaranteed that at least one cell is empty (.)

Output Format

The top-left cell of the map is at position (0, 0). Output two integers r and c , separated by a space, to indicate the row and column index (0-based) of the position where to place Skadi.

Note that:

- Skadi must be placed on an empty cell (.)
- If there are multiple positions that support the maximum number of operators, choose the one with the smallest row index r . If there is still a tie, choose the one with the smallest column

index c.

Technical Specification

- $1 \leq m, n \leq 1,000$

Sample Input 1

```
4 4
. **.
* . . .
* . . .
***.
```

Sample Output 1

```
2 1
```

Sample Input 2

```
3 6
..*...
.***..
..*...
```

Sample Output 2

```
0 1
```